ZeroJ Use Case Demos

ZeroJ Use Case Demos — Presentation

Zero-Knowledge Proofs on Cardano: From Circuit to On-Chain Verification

7 end-to-end demo applications demonstrating real-world ZK use cases

---

What is ZeroJ?

ZeroJ is a **pure Java ZK toolkit** for Cardano that enables:

- **Circuit definition** in Java (symbolic annotations or CircuitSpec DSL)
- **Proof generation** using Groth16 on BLS12-381 (pure Java, zero native dependencies)
- **On-chain verification** via Cardano Plutus V3 BLS12-381 builtins
- **Smart contracts** written in Java using Julc (Java → Plutus V3 compiler)

```
Java Circuit DSL → R1CS → Groth16 Prove (pure Java) → 192-byte proof → Plutus V3 Verify (on-chain)
```

Technology Stack (All Demos)

Layer	Technology
Backend	Spring Boot 3.5.0-M3, Java 25 (GraalVM)
ZK Proofs	ZeroJ — Groth16, BLS12-381, pure Java prover
On-chain	Julc → Plutus V3 validators
Cardano Client	cardano-client-lib 0.8.0
Frontend	Svelte 5 + Vite
Persistence	H2 (metadata), RocksDB (Merkle trees)
Local Devnet	Yaci DevKit
Proof Caching	Groth16SetupCache / PlonkSetupCache (binary SRS + setup serialization)

Demo Portfolio Overview

#	Demo	Port	What it Proves	On-Chain Pattern
1	**Private NFT Ownership**	8085	"I own an NFT from this collection"	Sorted linked list + nullifiers
2	**Private Voting**	8086	"I'm eligible AND my encrypted vote is 0 or 1"	Sorted linked list + homomorphic tally
3	**Identity/KYC**	8087	"I meet KYC requirements (age, country)"	Stateless spending validator
4	**Digital Product Passport**	8088	"Product meets EU compliance thresholds"	Minting policy (mint-if-compliant)
5	**Proof of Reserves**	8089	"Reserves >= liabilities, all balances >= 0"	Spending validator (attestation)
6	**Personhood Airdrop**	8083	"I'm a unique person and have not claimed this epoch"	Nullifier NFT
7	**Selective Disclosure**	8091	"One signed credential satisfies multiple private predicates"	Predicate-gated validators

Top-level directories are full Spring Boot E2E demos. Lightweight developer examples live under `examples/minimal-circuits/`.

Code Metrics

Metric	Value
Total Java source files	**70 files**
Total Java lines of code	**~8,200 lines**
Total frontend code (Svelte + TypeScript)	**~1,500 lines**
On-chain Plutus validators	**9 validators** across 5 demos
ZK circuits	**8 circuits** (threshold, membership, inspection, solvency, vote, credential, NFT ownership)
Proof size (all circuits)	**192 bytes** (constant)

Demo 1: Private NFT Ownership

Port: 8085 | **Problem**: Proving NFT ownership reveals your wallet address

What it Proves

> "I own an NFT from this collection — without revealing which wallet holds it"

Architecture

```
Holder → registers its token (CIP-8 signature from the holding address), gets a 252-bit
secret
Indexer → one snapshot per event, frozen at build; the event's root and context are
script parameters
Holder → ZK proof: "my leaf is in the snapshot, and this is my nullifier for this
event"
       → one transaction mints the nullifier token and inserts it into the sorted list
       → a second use of the same holding has the same nullifier → rejected
```

Key Components

Component	Details
Circuit	`NFTOwnershipProof` v2 — Poseidon Merkle proof (depth 10) and nullifier `Poseidon(Poseidon(secret, tokenName), contextId)`
On-chain	`ZkProofMintingPolicy` (Groth16; root and context as parameters) + `NullifierListValidator` (one-shot root, insert-only, key = the proved nullifier)
Pattern	Two minting policies coupled — the list admits exactly the nullifier the ZK policy minted

Demo Flow

1. Mint NFTs on Yaci DevKit (to the custody wallet)
2. Register holders → build and freeze the event snapshot
3. Generate the ZK proof
4. Submit on-chain → nullifier inserted in the sorted list
5. Use the same holding again → rejected (nullifier exists)

What's Hidden vs What's Revealed

Hidden	Revealed
Wallet address and which NFT was used	"Some registered holder of this collection acted"
Other token holdings	Nullifier, unlinkable to the token without the holder's secret

An earlier version used `Poseidon(tokenName, contextId)`, which anyone could recompute for every token, and its list did not bind the inserted key to the proved nullifier. See ADR-0009 D2.

Demo 2: Private Voting

Port: 8086 | **Problem**: DAO votes are public (enables vote buying, social pressure)

What it Proves

> "I'm an eligible voter AND my vote is valid (0 or 1) — without revealing who I am or how I voted"

Architecture

```
Trustees → 3 key shares, each with a proof of possession → election key PK = PK1 + PK2
+ PK3
Voter    → Ballot = ElGamal encryption of the vote under PK: A = [k]·G, B = [vote]·G +
[k]·PK
         → Prove: "I'm in the voter Merkle tree + the nullifier is mine + (A, B)
encrypts 0 or 1"
         → Nullifier = Poseidon(secretKey, electionId) — one vote per voter
         → Insert on-chain via sorted linked list (node = nullifier + ballot)
Tally    → after the deadline: sum the ballots while encrypted, each trustee posts a
           decryption share with a DLEQ proof, and only the total is decrypted, once
```

An earlier version stored `Poseidon(vote, nullifier)`. That hid the voter but not the vote: the nullifier is public and a vote is 0 or 1, so anyone could hash both and compare (issue #7, fixed by ADR-0005).

Key Components

Component	Details
Circuits	`PrivateBallotProof` (eligibility + nullifier + `ZkElGamal.encrypt` of a 1-bit vote, 9,477 constraints) and `TrusteeShareProof` (DLEQ for key and decryption shares)
On-chain	`VoteListValidator` (linked list) + `VoteZkMintingPolicy` (Groth16; election id, voter root, election key and deadline are script parameters)
Tally	`TallyService`: homomorphic sum, verified trustee shares, one decryption after the deadline, re-verifiable from the election manifest
Test accounts	5 voter accounts and 3 trustees auto-created at startup with Yaci DevKit funding

Demo Flow

1. Election auto-created with 5 voters and 3 trustees
2. voter1 votes YES → on-chain tx (encrypted ballot)
3. voter2 votes NO → on-chain tx (encrypted ballot)
4. voter1 tries again → **double vote rejected**
5. After the deadline: trustees decrypt the total → YES: 1, NO: 1

Demo 3: Identity / KYC

Port: 8087 | **Problem**: KYC requires revealing sensitive personal data

What it Proves

> "I have a valid credential AND age >= 18 AND my country is in the approved list — without revealing my age, country, or identity"

Architecture

```
Issuer → Sign Poseidon(age, country) with an EdDSA-Jubjub issuer key
User   → Generate ZK proof: registered-key signature + age check + country Merkle
       → On-chain: validator pins issuer/policy parameters and verifies Groth16
       → Verifier sees: "eligible: YES" — nothing else
```

Key Components

Component	Details
Circuit	`CredentialProof` — symbolic annotation circuit, EdDSA-Jubjub verify + 8-bit age comparison + country Merkle
On-chain	`CredentialGatedValidator` (spending validator, datum-based Groth16)
Pattern	**Stateless** — no nullifiers, proofs can be reused (ongoing DeFi access)
Credential	EdDSA-Jubjub-signed credential over Poseidon claims; the issuer key is random and persisted (ADR-0009 D11)

Test Users

User	Age	Country	Expected
Alice	25	USA	**ELIGIBLE**
Bob	30	GBR	**ELIGIBLE**
Charlie	16	USA	**NOT ELIGIBLE** (underage)
Diana	22	BRA	**NOT ELIGIBLE** (country)
Eve	45	JPN	**ELIGIBLE**

Key Difference

Unlike NFT/voting demos, this is **stateless** — no linked list, no nullifiers. The same proof can be used repeatedly for ongoing DeFi access, and a proof copied from the chain opens any other UTxO at the same gate: a real deployment binds the proof to the output it spends.

Demo 4: Digital Product Passport (DPP)

Port: 8088 | **Problem**: EU ESPR mandates product compliance data, but supply chain details are trade secrets

What it Proves

> "Product meets carbon/recycled/origin/inspection thresholds — without revealing exact values or supply chain details"

Architecture

```
Auditor      → measures, commits each claim with a Pedersen vector commitment (fresh
252-bit blinding)
             → gives the opening (value, blinding) to the manufacturer only
Manufacturer → proves each claim's rule against its commitment (Groth16)
             → builds one passport transaction; the auditor co-signs it once per
product
Cardano      → DppPassport mints exactly one token named productId into a locked output
whose
               datum holds every attested commitment; only the auditor can revoke
(burn) it
```

Key Innovations

1. Auditor-issued Pedersen commitments (ADR-0009 D1)

- The published commitment hides each measurement: a blinding of 252 random bits makes the 65,536-value search that broke the old design learn nothing.
- The manufacturer can't choose its own value or threshold: the auditor commits, the script pins the threshold.
- The passport keeps the commitments, so a claim can be re-proved against a stricter threshold, or opened to a regulator, without asking the auditor again.

2. One circuit per claim

Circuit	Rule	Constraints
`CarbonClaimProof`	carbon ≤ threshold	2,192
`RecycledClaimProof`	recycled ≥ threshold	2,192
`OriginClaimProof`	country in the EU set (27 states)	3,344
`InspectionClaimProof`	3 inspections by approved inspectors, in time order	5,094

3. Locked passport record

- Minting checks both signatures, the exact mint map, the datum and the carbon proof (27.9 % of the per-transaction CPU budget).
- The passport never moves and its datum never changes; the auditor alone can revoke it.

4. Poseidon MPF registry

- Custom `PoseidonCommitmentScheme` for CCL's MPF, persisted in RocksDB; it stores `productId → passport outref` only, never measurements.

The earlier version hashed each measurement with a secret every manufacturer had to hold (a published constant in the demo), so any 16-bit measurement fell to brute force from the mint datum. See ADR-0009 F1 and D1.

Demo Scenarios

Product	Carbon	Recycled	EU	Inspections	Result
BAT-SN001 (battery)	7kg ✓	45% ✓	DEU ✓	3/3 ✓	**ALL COMPLIANT** → passport minted
BAT-SN003 (battery)	65kg ✗	20% ✗	DEU ✓	3/3 ✓	**NOT COMPLIANT** → no proof exists, no passport
TEX-B2024-001 (100 t-shirts)	3kg ✓	42% ✓	FRA ✓	—	**ALL COMPLIANT**
TEX-B2024-003 (500 units)	15kg ✓	5% ✗	CHN ✗	—	**NOT COMPLIANT**

Demo 5: Proof of Reserves

Port: 8089 | **Problem**: Exchanges collapse (FTX) — users can't verify solvency without seeing all balances

What it Proves

> "Total reserves >= total liabilities AND all account balances >= 0 — without revealing any individual balance"

Architecture

```
Exchange → Register accounts with balances (H2)
         → Build the liabilities tree: leaf = Poseidon(Poseidon(idTag, salt), balance), fresh salt per snapshot
         → Generate solvency proof (Groth16): the leaves sum to the total, and reserves ≥ total
         → Mint the period's attestation token (Plutus V3 verifies the proof at mint; locked datum)
User     → Gets its salt and Merkle path, recomputes the root in the browser
         → Compares it with the attestation read from the chain indexer, not the exchange's API
```

Key Innovation: proof at mint, salted leaves

- The attestation exists only as a token whose minting verified a solvent statement, so a datum alone proves nothing.
- Each leaf absorbs a 252-bit salt, so a customer's path no longer reveals a neighbour's balance (ADR-0009 D4). The circuit sums every leaf; internal nodes are plain Poseidon hashes.

Circuit

Component	Details
`SolvencyProof`	Symbolic annotation circuit with Merkle tree reconstruction + 64-bit range checks + solvency comparison
Range checks	All balances `assertInRange(64)` — prevents negative balance cheating
Sum verification	Computed sum must match published total liabilities
Solvency	`reserves >= totalLiabilities` (64-bit comparison)

Demo Scenarios

Scenario	Reserves	Liabilities	Result
Solvent	10,000 ADA	8,500 ADA	✅ **SOLVENT** — Groth16 verified on-chain
Insolvent	5,000 ADA	8,500 ADA	❌ **INSOLVENT** — no attestation can be minted
User verify	—	—	✅ Alice's 500 ADA included, checked against the on-chain root

---

On-Chain Patterns Summary

Each demo showcases a different Cardano UTXO pattern:

Pattern	Used By	How it Works
Sorted Linked List	NFT, Voting	Each entry is a UTXO. Sorted insertion proves non-existence. Natural concurrency.
Stateless Spending	Identity/KYC	Lock ADA → unlock with ZK proof. No state growth. Proof reusable.
Locked record	DPP	An auditor-co-signed passport minted with its proof into a script output that only the auditor can burn.
Attestation token	Proof of Reserves	One token per period, minted only with a solvent proof, locked until the period ends.

Performance

Metric	Value
Proof size	**192 bytes** (constant for all circuits)
Proof generation (simple circuit, ~1,700 constraints)	**3-8 seconds**
Proof generation (complex circuit, ~10,800 constraints)	**15-30 seconds**
On-chain verification cost	**~0.3-0.5 ADA** per Groth16 pairing check
Startup with proof-system setup cache	**~14 seconds** (vs ~60s cold start)
Circuit compilation	**< 1 second**

Security Properties (Across All Demos)

Property	How it's Achieved
Zero knowledge	Groth16 proof reveals nothing about secret inputs
Soundness	BLS12-381 pairing check — computationally infeasible to forge
On-chain enforcement	Plutus V3 validators verify proofs — can't bypass by using a different UI
Anti-replay	Nullifiers (NFT, Voting), stateless reuse (KYC, by design), auditor co-signature once per product (DPP)
No guessable published values	Every published hash or commitment of a small value (vote, measurement, balance, identifier) also absorbs ≥ 252 secret random bits (ADR-0009 I1)
Demo custody	The demo servers hold users' keys; their APIs are the operator's view. Privacy is claimed against chain and mempool observers.
Persistence	H2 + RocksDB + proof-system setup cache — survives restarts
Deterministic setup	Same tau → same SRS → same policy IDs across restarts

Running the Demos

Prerequisites

```
sdk install java 25.0.2-graal   # Java 25 (GraalVM)
yaci-cli devkit start            # Local Cardano devnet
```

Start Any Demo

```
sdk use java 25.0.2-graal
cd zeroj-usecases/<demo-name>

# Top up admin wallet
curl -X POST http://localhost:10000/local-cluster/api/addresses/topup \
  -H "Content-Type: application/json" \
  -d
        '{"address":"addr_test1qryvgass5dsrf2kxl3vgfz76uhp83kv5lagzcp29tcana68ca5aqa6sw
        lq6llfamln09tal7n5kvt4275ckwedpt4v7q48uhex","adaAmount":10000}'

# Build and run
./gradlew clean build -x test
java --enable-native-access=ALL-UNNAMED -jar build/libs/<demo-name>-0.1.0-SNAPSHOT.jar
```

Demo URLs

Demo	URL
NFT Ownership	http://localhost:8085
Private Voting	http://localhost:8086
Identity/KYC	http://localhost:8087
Digital Product Passport	http://localhost:8088
Proof of Reserves	http://localhost:8089

Repository Structure

```
zeroj-usecases/
├── CLAUDE.md                          # Shared tech stack NFRs
├── PRESENTATION.md                    # This document
├── nft-ownership/                     # Demo 1: Private NFT Ownership
│   ├── README.md
│   ├── DESIGN.md                      # Detailed technical design
│   ├── src/main/java/.../nft/
│   │   ├── circuit/                   # ZK circuit definition
│   │   ├── onchain/                   # Julc → Plutus V3 validators
│   │   ├── service/                   # Business logic + Cardano integration
│   │   └── controller/               # REST API
│   └── frontend/                      # Svelte 5 + Vite
├── private-voting/                    # Demo 2: Private Voting
├── identity-kyc/                      # Demo 3: Identity/KYC
├── digital-product-passport/          # Demo 4: DPP
│   ├── docs/mpf-architecture.md       # Poseidon MPF design
│   └── ...
└── proof-of-reserves/                 # Demo 5: Proof of Reserves
```

Key Takeaways

1. **ZK proofs are practical on Cardano today** — pure Java prover, Plutus V3 BLS12-381 builtins, ~192 byte proofs
2. **Multiple UTXO patterns** demonstrated — sorted linked lists, stateless validators, minting policies, attestation UTXOs
3. **Real-world use cases** — NFT privacy, DAO governance, KYC compliance, EU DPP regulation, exchange solvency
4. **Production path clear** — MPC ceremonies replace dev setup, circuits are the same, on-chain verifiers are identical
5. **Full persistence** — H2 + RocksDB + proof-system setup cache means demos survive restarts and scale realistically

---

Future Work

- **Credential envelopes**: Define a reviewed DID/VC profile around the suite-specific EdDSA-Jubjub credential rather than claiming Ed25519 interoperability
- **Private Token Transfer**: Privacy pool (deposit/withdraw) — Cardano's first mixer
- **Recursive proofs**: Aggregate multiple proofs into one (for DPP supply chains, large reserve proofs)
- **MPF in-circuit verification**: `SignalMpf` circuit gadget for full Poseidon MPF proof verification inside ZK
- **Production MPC**: Integrate Hermez/Perpetual Powers of Tau ceremony outputs

Built with ZeroJ — Pure Java ZK for Cardano